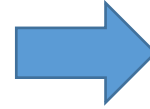


Feature Representation

Washington County jail *served*
11,166 meals last month - a figure
that translates to feeding some 120
people three times daily for 31 days

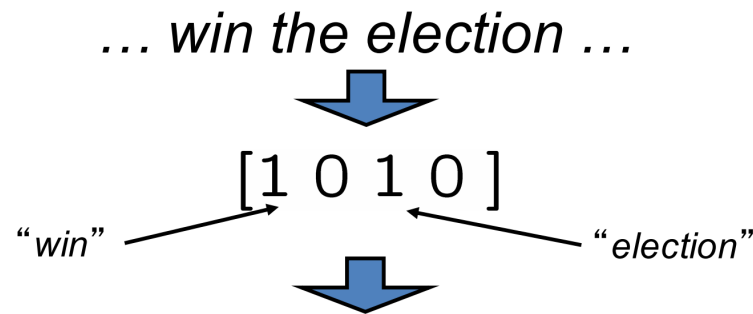


- Features are indicator functions which count the occurrences of certain patterns in the input
- Different feature values for every pair of input x and class y

context:jail = 1
context:county = 1
context:feeding = 1
context:game = 0
...
local-context:jail = 1
local-context:meals = 1
...
object-head:meals = 1
object-head:ball = 0

Block Notation

- We often think of the feature function as a mapping from a pair of input and label pair to a feature vector
 - In these cases, the feature vector will take a block form, as below



$$\begin{aligned}\phi(x, \textit{SPORTS}) &= [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \phi(x, \textit{POLITICS}) &= [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \phi(x, \textit{OTHER}) &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0]\end{aligned}$$

Prediction

- In a linear model, each feature gets a weight
 - Feature function: ϕ
 - Weight vector: w

$$\begin{aligned}\phi(x, \textit{SPORTS}) &= [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \phi(x, \textit{POLITICS}) &= [0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \hline w &= [1 \ 1 \ -1 \ -2 \ 1 \ -1 \ 1 \ -2 \ -2 \ -1 \ -1 \ 1]\end{aligned}$$

- The prediction of y is the value that maximizes the score

$$\begin{aligned}\phi(x, \textit{SPORTS})^T w &= 1 * 1 + (-1) * 1 = 0 \\ \phi(x, \textit{POLITICS})^T w &= 1 * 1 + 1 * 1 = 2\end{aligned}$$



Prediction(x) = *POLITICS*

Log-linear Models

- Probabilistic model: use the scores as probabilities:

$$p(y|x; w) = \frac{\exp(w \cdot \phi(x, y))}{\sum_{y'} \exp(w \cdot \phi(x, y'))}$$

score(x,y,w) 

- Prediction: straightforward. Compute $P(y/x)$ for each possible label, and select the maximum

Log-linear Models

- Learning: maximize the (log) conditional likelihood of training data

$$L(w) = \log \prod_{i=1}^n P(y_i | x_i; w) = \sum_{i=1}^n \log P(y_i | x_i; w)$$

$$w^* = \arg \max_w L(w)$$

Log-linear Models

- The conditional likelihood is concave, which means that we can maximize it using standard convex optimization techniques
 - Like gradient ascent or quasi-Newton methods
 - The gradient is given as:

$$L(w) = \sum_{i=1}^n \log P(y_i | x_i; w) \quad P(y | x; w) = \frac{e^{w \cdot \phi(x, y)}}{\sum_{y'} e^{w \cdot \phi(x, y')}}$$

$$\frac{\partial}{\partial w_j} L(w) = \sum_{i=1}^n \left(\phi_j(x_i, y_i) - \sum_{y'} P(y' | x_i; w) \phi_j(x_i, y') \right)$$

Total count of feature j
in correct candidates

Expected count of
feature j in predicted
candidates

Derivation

$$L(w) = \sum_{i=1}^n \log \left(\frac{e^{w \cdot \phi(x, y)}}{\sum_{y'} e^{w \cdot \phi(x, y')}} \right) =$$
$$\sum_{i=1}^n \left(w \cdot \phi(x, y) - \log \left(\sum_{y'} e^{w \cdot \phi(x, y')} \right) \right)$$

$$\frac{\partial L(w)}{\partial w_j} = \sum_{i=1}^n \left(\phi_j(x, y) - \frac{1}{\sum_{y'} e^{w \cdot \phi(x, y')}} \cdot \sum_{y''} \phi_j(x, y'') \cdot e^{w \cdot \phi(x, y'')} \right) =$$
$$\sum_{i=1}^n \left(\phi_j(x, y) - \sum_{y''} \phi_j(x, y'') \cdot \frac{e^{w \cdot \phi(x, y'')}}{\sum_{y'} e^{w \cdot \phi(x, y')}} \right) =$$
$$\sum_{i=1}^n \left(\phi_j(x, y) - \sum_{y''} P(y''|x) \cdot \phi_j(x, y'') \right)$$

Regularization

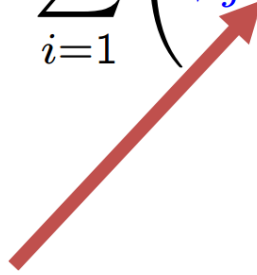
- The log-linear model doesn't have the same issue with zero counts as the generative model
- However, it may overfit the data by inflating w
 - If a certain feature appeared once with the class *SPORTS*, the model would have an incentive to place a very high weight for that feature and the label *SPORTS*
 - To combat this, we add a regularization term (often l_2 regularization)

$$L(w) = \sum_{i=1}^n \log p(y_i|x_i; w) - \frac{\lambda}{2} ||w||^2$$

Regularization: Modified Gradient

$$L(w) = \sum_{i=1}^n \left(w \cdot \phi(x_i, y_i) - \log \sum_y \exp(w \cdot \phi(x_i, y)) \right) - \frac{\lambda}{2} ||w||^2$$

$$\frac{\partial}{\partial w_j} L(w) = \sum_{i=1}^n \left(\phi_j(x_i, y_i) - \sum_y p(y|x_i; w) \phi_j(x_i, y) \right) - \lambda w_j$$



Total count of feature j
in correct candidates



Expected count of
feature j in predicted
candidates

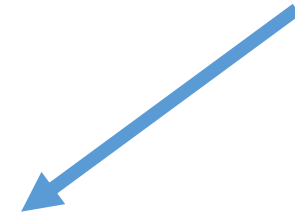


Big weights
are bad

SGD for Log-Linear Models and Perceptron

1. $w \leftarrow \vec{0}$
2. **for** $r = 1 \dots N_{iterations}$
3. **for** $i = 1 \dots N$
4. $w \leftarrow w + \eta(\phi(x_i, y_i) - \mathbb{E}_{\hat{y}}[\phi(x_i, \hat{y})])$
5. **return** w

In Perceptron, a maximum rather than expectation



Example: Named Entity Recognition

- The task: finding all the names mentioned in a text and classifying them into their types (e.g., *Location, Organization, Person, Other*)
- Example:

Citing high fuel prices, [ORG **United Airlines**] said [TIME **Friday**] it has increased fares by [MONEY **\$6**] per round trip on flights to some cities also served by lower-cost carriers. [ORG **American Airlines**], a unit of [ORG **AMR Corp.**], immediately matched the move, spokesman [PER **Tim Wagner**] said. [ORG **United**], a unit of [ORG **UAL Corp.**], said the increase took effect [TIME **Thursday**] and applies to most routes where it competes against discount carriers, such as [LOC **Chicago**] to [LOC **Dallas**] and [LOC **Denver**] to [LOC **San Francisco**].

Example: Regularization in NER

Because of regularization,
the more common prefixes
have larger weights even
though entire-word features
are more specific

Local Context

	Prev	Cur	Next
Word	at	Grace	Road
Tag	IN	NNP	NNP
Sig	x	Xx	Xx

Feature Weights

Feature Type	Feature	PER	LOC
Previous word	at	-0.73	0.94
Current word	Grace	0.03	0.00
Beginning bigram	Gr	0.45	-0.04
Current POS tag	NNP	0.47	0.45
Prev and cur tags	IN NNP	-0.10	0.14
Current signature	Xx	0.80	0.46
Prev-cur-next sig	x-Xx-Xx	-0.69	0.37
P. state - p-cur sig	O-x-Xx	-0.20	0.82
...			
Total:		-0.58	2.68